Digital Signal Processing – Exercise Outline

Engine signals as the guiding example, run on Wokwi and JSLinux

Draft, November 12, 2026

Concept

Every exercise sheet follows the leitmotif of the lecture: *The ECU does not see the engine. It sees only signals.* The exercises therefore build, chapter by chapter, a small *virtual engine* (signal generators for crankshaft, camshaft, knock, ion current and lambda sensors) and a small *virtual ECU* that extracts information from these signals. Sheet 12 (knock detection) assembles all parts.

Platforms

[Wokwi] Browser-based microcontroller simulator (`https://wokwi.com`). Targets: Arduino Uno (8-bit, no FPU, 10-bit ADC – for fixed-point and resource-limit exercises) and ESP32 (FPU, 12-bit ADC – for floating-point real-time filtering). Inputs via potentiometers, buttons and *Wokwi custom chips* (C API) that act as sensor signal generators; outputs via Serial Monitor (CSV), LEDs and the logic analyzer (VCD export).

[JSLinux] x86 Linux in the browser (`https://bellard.org/jslinux`). Plain C99 with `libm` (`gcc` or `tcc`), offline analysis of recorded or synthetic data, text/ASCII plots and CSV output. Data transfer from Wokwi: copy the Serial Monitor CSV, upload into JSLinux.

[Paper] Pen-and-paper part (derivations, sketches, short answers), used to prepare or check the programming part.

Course infrastructure (to be provided once)

- `engine_signals.h/.c`: portable C models of all sensor signals (60-2 crank wheel, cam, knock = damped resonances + noise, ion current, switching lambda sensor), shared by Wokwi and JSLinux.
- Wokwi custom chips `crank-60-2`, `knock-sensor`, `lambda-probe` built from the same models, with RPM/load/knock-intensity controls.
- `asciiplot.h`: dependency-free terminal plot for JSLinux (time signal, spectrum, pole-zero map).
- `csvio.h`: read/write of sample files; a set of pre-recorded data files (crank, knock with/without knock, lambda).
- Sheet 0 (*Getting started*, 45 min): blink + serial CSV and the engine simulator chip on Wokwi; compile/run/upload/download on JSLinux as an optional bonus exercise. The JSLinux bonus of Sheet 2 repeats the setup in a short section; Sheet 5 (first mandatory JSLinux sheet) starts with the same setup section for everybody. Sheets 3 and 4 run their C programs on a Wokwi ESP32 without circuit.

Sheet format

Each sheet: learning goals referencing the script, 1–2 paper exercises, 2–3 lab exercises, deliverables (code, plot, short written answers), optional challenge. Solutions in the same `.tex` file, switched by `\ifsolutions`. Expected workload per sheet: 90 min tutorial + about 3 h self-study.

Overview

#	Chapter	Lab core	[Wokwi]	[JSLinux]
1	ECU and Signals	60-2 crank signal, RPM from tooth timing (bonus: signal zoo)	x	(x)
2	Systems	Lambda loop with delay (bonus: black-box property tests, engine dynamics)	x	(x)
3	LTI Systems	Convolution, impulse/step response, moving-average RPM	x	
4	Sampling	ADC sampling, quantisation SNR, time vs. angle domain	x	
5	Nyquist	Aliasing of knock frequencies, anti-aliasing filter	x	x
6	Fourier	Fourier series of crank signal, engine orders		x
7	Laplace	Knock sensor as resonator, RC filter, step responses		x
8	DFT/FFT	Own radix-2 FFT, windowing, leakage, cost	x	x
9	Z-Transform	Pole/zero, stability, digital resonator	x	x
10	Digital Filters	FIR/IIR knock band-pass, fixed point vs. float	x	x
11	Maximum Entropy	Burg/AR spectrum vs. FFT on short windows		x
12	Knock Detection	Angle-synchronous real-time knock detector	x	x

(x) = optional bonus exercise only. Sheets 0–4 are Wokwi-only (Sheets 3 and 4 run their C programs on an ESP32 without circuit); JSLinux becomes mandatory from Sheet 5 on.

1 Sheet 1 – The ECU Sees Only Signals

Goals: describe sensor signals mathematically, classify them, extract a first piece of information (engine speed) from a raw signal.

1.1 Signal classification [Paper]

- Classify the five engine signals (continuous/discrete, deterministic/stochastic, periodic/aperiodic, analog/digital) and justify each entry (sensors of Table 1.1); discuss borderline cases (*quasi-periodic* knock, crank signal during acceleration).
- Relation crank angle vs. cam angle (720° vs. 360°): period of each signal at 3000 rpm.

1.2 60-2 crank wheel and engine speed [Wokwi]

- Arduino Uno + `crank-60-2` chip: capture rising edges by interrupt, store tooth periods, print CSV.
- Detect the missing-tooth gap (period ratio > 2), count teeth, compute crank angle and RPM; light an LED at TDC.
- Vary RPM with a potentiometer; observe jitter in the RPM estimate.

1.3 Cam/crank synchronisation (challenge) [Wokwi]

- Add the cam signal and determine the 720° cycle phase (cylinder identification).

1.B1 Signal zoo (bonus, optional) [JSLinux]

- Generate and plot all five sensor models with `engine_signals.h`; compute mean, RMS, min/max.
- Determine numerically whether a signal is periodic (autocorrelation peak) – compare crank at constant speed vs. knock signal.

2 Sheet 2 – Systems

Goals: the operator view $y = S\{x\}$; static/dynamic, causal/non-causal, memoryless/with memory; first contact with linearity and time invariance. Wokwi-only; the JSLinux exercises are an optional bonus at the end, with a short JSLinux setup section (for students who skipped the bonus exercises of Sheets 0–1). Exercise 2.2 explains the two-point controller in detail.

2.1 Properties by hand [Paper]

- For about 8 systems (e.g. $y = 2x$, $y = x^2$, $y[n] = x[n+1] - x[n]$, three-point moving average, $y(t) = x(t - T_d)$, rectifier, integrator, $y[n] = n\,x[n]$) decide static/dynamic, causal, memory, linear, time-invariant.

2.B1 Black-box system identification (bonus) [JSLinux]

- Provided object file with five unknown systems `S1..S5(x, y, N)`. Students write test harnesses: superposition test, shifted-input test, causality test (impulse at n_0), memory test.
- Report: property table with numerical evidence.

2.B2 Engine as a dynamic system (bonus) [JSLinux]

- Simulate crankshaft speed as first-order system (inertia) driven by fuel quantity; step in fuel → gradual RPM change.
- Lambda path: fuel → engine → transport delay → switching lambda sensor (non-linear, static).

2.2 Closed lambda loop [Wokwi]

- ESP32 + `lambda-probe` chip: implement a two-point (bang-bang) controller with integrator; observe the limit cycle of Fig. 1.3 in the Serial Monitor; vary the transport delay and explain the change of the oscillation period.

3 Sheet 3 – LTI Systems

Goals: impulse response, convolution, step response, difference equations; LTI systems as the workhorse of DSP.

3.1 Convolution by hand [Paper]

- Graphical and tabular convolution of short sequences; impulse/step response of moving average and of $y[n] = a\,y[n-1] + (1-a)\,x[n]$.

3.2 Convolution engine [Wokwi]

- Implement `conv()` and verify commutativity, associativity and distributivity numerically; compare with recursive implementation of the exponential smoother.
- Cascading: two moving averages = triangular impulse response.

3.3 Smoothing the RPM estimate [Wokwi]

- Extend Ex. 1.3: moving average over N teeth vs. exponential smoother; trade-off noise vs. delay during an RPM ramp; measure the lag in teeth.
- Why must the gap tooth be treated separately (violation of time invariance in the tooth domain)?

4 Sheet 4 – Sampling

Goals: sampling and quantisation, ADC resolution, SNR, timing jitter, time-based vs. crank-angle-based sampling.

4.1 Quantisation [Paper]

- Derive SNR $\approx 6.02N + 1.76\,\text{dB}$; required ADC bits for the knock sensor dynamic range; LSB size for 10-/12-bit ADCs.

4.2 ADC in practice [Wokwi]

- Uno/ESP32: timer-driven sampling of a potentiometer and of the `knock-sensor` chip; verify the sampling rate with the logic analyzer (toggle pin per sample).
- Note: Wokwi runs in simulated time; timing is measured in simulated time, not wall-clock time.

4.3 Quantisation noise and jitter [Wokwi]

- Quantise a sine with $N = 4 \ldots 16$ bits; measure SNR, compare with the formula; add sampling jitter and observe the SNR drop vs. frequency. ESP32 program without circuit.

4.4 Time domain vs. angle domain [Wokwi]

- Resample a time-sampled signal during an RPM ramp onto a crank-angle grid (linear interpolation using crank edges); show that crank-synchronous components become stationary in the angle domain. ESP32 program without circuit.

5 Sheet 5 – Nyquist

Goals: sampling theorem, aliasing, reconstruction, anti-aliasing filters.

5.1 Aliasing by hand [Paper]

- Alias frequency of knock components (6–15 kHz) for several sampling rates; minimum f_s; folding diagram.

5.2 Aliasing made visible [Wokwi]

- Uno ADC (max. about 9 kHz achievable) samples the knock chip at 6, 8, 11 kHz resonance: observe the apparent frequency in the CSV; switch to ESP32 with higher f_s.

5.3 Reconstruction and anti-aliasing [JSLinux]

- Sinc interpolation vs. zero-order hold vs. linear interpolation; reconstruction error.
- Simulate an RC (and 2nd-order) anti-aliasing filter before sampling; residual alias power vs. filter order.

6 Sheet 6 – Fourier

Goals: Fourier series, Fourier transform, DTFT; engine orders.

6.1 Fourier series of engine signals [Paper]

- Fourier coefficients of a rectangular tooth signal; effect of the missing teeth on the spectrum (once-per-revolution modulation).
- Damped sinusoid (knock) $\rightarrow$ Lorentzian spectrum; bandwidth vs. damping.

6.2 Numerical Fourier series and DTFT [JSLinux]

- Compute Fourier coefficients numerically; Gibbs phenomenon for partial sums; DTFT evaluated on a fine grid (no FFT yet).

6.3 Engine orders [JSLinux]

- Signal with 2nd and 4th engine order and a fixed structural resonance: spectra at 1500/3000/6000 rpm; distinguish speed-proportional orders from fixed resonances.

7 Sheet 7 – Laplace

Goals: transfer functions, poles and zeros, stability and step response of continuous-time systems; bridge to discretisation.

7.1 Sensor models in the s-domain [Paper]

- Knock sensor as 2nd-order resonator $H(s)$ (resonance frequency, damping, Q); RC low-pass; lambda sensor as first-order lag + delay (e^{-sT_d}, Padé approximation).
- Poles, step response, Bode sketch.

7.2 Numerical verification [JSLinux]

- Simulate the ODEs (Euler, RK4) and compare with analytic step and impulse responses; evaluate $|H(j\omega)|$ and phase on a grid.
- Discretise by forward Euler, impulse invariance and bilinear transform; compare frequency responses (preview of Sheets 9/10).

8 Sheet 8 – DFT and FFT

Goals: DFT as sampled DTFT, frequency resolution, leakage, windows, FFT algorithm and complexity.

8.1 DFT properties [Paper]

- 4- and 8-point DFT by hand; resolution $\Delta f = f_s/N$; zero padding vs. true resolution; window main lobe/side lobe.

8.2 Own FFT [JSLinux]

- Implement direct DFT and iterative radix-2 FFT; verify against each other; measure runtime vs. N and fit N^2 vs. $N \log N$.
- Rectangular, Hann, Hamming, Blackman windows: leakage of a non-bin-centred knock frequency.

8.3 FFT on the microcontroller [Wokwi]

- ESP32: 256-point FFT of the knock chip signal in a crank-angle window, print the spectrum, identify the knock resonance.
- Uno: memory limit – largest feasible N; operation count instead of execution time (Wokwi is not cycle-accurate).

9 Sheet 9 – Z-Transform

Goals: z-transform, ROC, transfer function $H(z)$, poles/zeros, stability, frequency response from the pole-zero map.

9.1 Analytical [Paper]

- Transforms of standard sequences; inverse by partial fractions; $H(z)$ of the exponential smoother and of the moving average; stability criterion.

9.2 Pole-zero explorer [JSLinux]

- Program that reads $H(z)$ coefficients, computes poles/zeros (quadratic formula/Durand–Kerner), ASCII pole-zero plot, magnitude and phase response, impulse response.

9.3 Digital resonator for knock frequency [Wokwi]

- Two-pole resonator at the knock frequency; pole radius r vs. bandwidth; Uno with float vs. Q15 fixed point: observe instability/limit cycles for $r \to 1$.

10 Sheet 10 – Digital Filters

Goals: FIR (window method) and IIR (bilinear transform) design, structures (direct form, biquad cascade), coefficient quantisation.

10.1 Design by hand [Paper]

- Specification of a knock band-pass (e.g. 6–8 kHz, $f_s = 40$ kHz); FIR order estimate; 2nd-order IIR band-pass via bilinear transform with pre-warping.

10.2 Filter design tool [JSLinux]

- Implement FIR window design and IIR biquad design; verify with the Sheet 9 explorer; export coefficients as C header (float and Q15).
- Coefficient quantisation: pole shift and response change for 8/12/16-bit coefficients.

10.3 Real-time filtering [Wokwi]

- ESP32: knock band-pass (FIR and biquad) on the knock chip signal, sample-by-sample in the timer ISR; compare outputs.
- Uno: Q15 biquad; overflow handling and scaling.

11 Sheet 11 – Maximum Entropy

Goals: autoregressive (AR) modelling, Yule–Walker, Levinson–Durbin, Burg method; high-resolution spectra from short records.

11.1 AR models [Paper]

- Yule–Walker equations for AR(1), AR(2); relation of AR(2) poles to a resonance; maximum-entropy principle.

11.2 MEM vs. FFT on short windows [JSLinux]

- Implement Levinson–Durbin and Burg; spectra of 32/64/128-sample knock windows vs. windowed periodogram.
- Model order selection (FPE/AIC); spurious peaks and line splitting for too high orders.

11.3 Resolving two resonances [JSLinux]

- Two knock modes close in frequency: minimum record length for resolution with FFT vs. Burg; influence of SNR.

12 Sheet 12 – Knock Detection (capstone)

Goals: combine all previous parts into a real-time, angle-synchronous knock detector; evaluate it statistically.

12.1 Detector design [Paper]

- Signal chain: crank sync → measurement window (e.g. 10–70° after TDC) → band-pass → rectification/squaring → integration → normalisation by background level → threshold. Justify each block with the respective chapter.

12.2 Offline evaluation [JSLinux]

- Run the detector on recorded data sets with known knock labels; ROC curve, choice of threshold; compare energy-based, FFT-based and MEM-based features.

12.3 Real-time knock detector [Wokwi]

- ESP32 + `crank-60-2` + `knock-sensor` chips: full chain in real time, knock LED per cylinder, ignition retard as output; vary RPM and knock intensity.
- Optional: port to Uno in fixed point and report the limits.

Open points

- Confirm the JSLinux image and toolchain to be used (`gcc` vs. `tcc`; Python is not assumed).
- Confirm which Wokwi features students may use (custom chips, ESP32, project sharing) under the chosen account type.
- Fix the numerical parameters (knock frequencies, sampling rates) once Chapters 4, 5 and 12 of the script are available.
- Language of the sheets (English, as the script).